Work Notes: 2 Nov 2026 (Lead sources, October traders, calls after conversion, shared IPs)

Nov 2, 2026 · @tanvik

Overview

This page covers everything after the 31 Oct notes. Most of it was one SQL report that grew step by step: calls made to converted clients, per client, from a pasted list of IDs, by the Sales team, with a 30-day window.

#	Task	Tool	Key idea to learn
1	Mastery workbook	Python	115 self-checking exercises on 29 to 31 Oct
2	Lead source for depositors	SQL	`DISTINCT ON` to pick one lead per customer
3	Who traded in October	pandas	`isin` + `np.where` flag
4	Manager pivot of traded clients	pandas	`nunique` on an ID kept only for traders
5	Calls after conversion	SQL	One conversion date per customer; `FILTER` buckets; `STRING_AGG`
6	Starting from an ID list	SQL	`UNNEST(ARRAY[...])` keeps every pasted ID
7	Team, Sales filter, 30-day columns	SQL	Join a saved question safely; filter on the joined table
8	Two misread requests	Process	Agree the output columns before rewriting
9	IDs sharing an IP	Metabase	The editor can't list values; convert to SQL

The end-of-day review comes first, then one section per task, then the problems log, cheat sheet and open items.

End-of-day review (day 4 of 7)

You asked 18 questions (10 SQL, 3 pandas, 3 Metabase, 2 about the workbook), and the calls-after-conversion report took 9 of them. The SQL habits from last week stuck: every join that could repeat rows was guarded. Most of the extra rounds came from the output columns not being pinned down at the start.

Solved today

- The lead source (Level 1 and Level 2) for every depositing customer, one lead per customer.
- An October traded / not traded flag on the filtered customers, and a Manager by Assigned To pivot of total and traded clients.
- A per-client report of Sales calls after conversion, with day buckets, call lists, 30-day counts, agents and TLs, starting from a pasted ID list.
- IPs shared by several Admin IDs in the Tdx Activity table.

Not confirmed yet

- Whether the team list has agents listed twice (the `HAVING COUNT(*) > 1` check).
- The exact Sales value in the team column (`'Sales'` vs a variant).
- Workbook progress: no `progress()` output shared yet.

Patterns

Pattern	Earlier	Today
Guarding against repeated rows	Leads repeated after joining status history (29 Oct)	Applied by default: one conversion date per customer, one team row per agent
Unclear output	—	The 30-day request went through day-by-day columns, then too few columns, before landing on two added columns
Fragile inputs	Wrong folder (31 Oct)	1,000+ IDs pasted into the query by hand

On the 30-day request, I misread it twice. Naming the exact columns you want ("add `calls_within_30_days` and `answered_within_30_days`, keep everything else") would have saved three rounds, and I should have asked before rewriting.

Gaps to study

- **Aggregates with `FILTER`:** one pass, many conditional counts.
- **`COUNT(*)` vs `COUNT(column)`:** rows versus non-blank values; this is what makes answered counts work.
- **Joining lookup tables safely:** `DISTINCT ON` or `GROUP BY` before the join.
- **Metabase limits:** what the editor can do, and when to switch to SQL.

Study programme check

No study chats since 29 Oct, and PG40 Day 1 is still at 0 of 40 in the chats found. The mastery workbook built on 31 Oct targets exactly this week's mistakes and checks itself, so it's the better place to start.

Focus for tomorrow

Mastery workbook, exercises 1 to 15, then run `progress()` and paste the result.

1. The mastery workbook

`Mastery_Workbook_29-31Oct.zip` holds 115 exercises that check themselves, with notes and a worked example per module, run on practice files shaped like the real exports. Every reference solution was tested from a freshly unzipped folder.

File	Role
`Mastery_Workbook.ipynb`	Notes, examples and exercises; start here
`Mastery_Solutions.ipynb`	One answer per exercise; open only after trying
`practice_data.py`	Builds the practice files on first run
`checker.py`	`check(n, ans)` marks an answer; `progress()` shows the score

```
pip install pandas openpyxl xlsxwriter polars pyarrow duckdb
```

Not covered yet

- Excel formulas: anchored `SUMIFS`, `REGEXEXTRACT`, OR inside `FILTER`, quotes, `COUNTIF` wildcards.
- Fetching from an API: paging by day, `raise_for_status()`, `json_normalize`.
- Smaller items: `read_excel(sheet_name=...)`, Polars `read_excel` and streaming collect, the pivot key-clash error, comparing old and new report versions.

Today's SQL (calls after conversion, ID lists, team joins) isn't in it either. A Module 12 could cover all of these.

2. SQL: lead source for depositing customers

Pick one lead per customer with `DISTINCT ON`, look up its source and that source's parent, then left-join the result onto the depositors. Joining `leads` directly would repeat a depositor once per lead.

```
WITH lead_source AS (
    SELECT DISTINCT ON (l.customer_id)
           l.customer_id,
           l.id                          AS lead_id,
           l.date_of_lead,
           COALESCE(s2.name, 'Unknown')  AS lead_source_1,
           s1.name                       AS lead_source_2
    FROM leads l
    LEFT JOIN sources s1 ON l.source_two_id = s1.id
    LEFT JOIN sources s2 ON s1.parent_id = s2.id
    WHERE l.customer_id IS NOT NULL
      AND l.deleted_at IS NULL
    ORDER BY l.customer_id, l.date_of_lead, l.created_at, l.id     -- first
lead
)
SELECT ctp.*,
       COALESCE(ls.lead_source_1, 'Unknown') AS lead_source_1,
       ls.lead_source_2, ls.lead_id, ls.date_of_lead
FROM customer_trading_profiles ctp
LEFT JOIN lead_source ls ON ls.customer_id = ctp.customer_id
WHERE ctp.total_deposit_count > 0;
```

How it works

- **Two joins to the same table:** `sources s1` is the lead's own source; `sources s2` is s1's parent (`s1.parent_id = s2.id`). Aliases let one table play two roles.
- **`DISTINCT ON (customer_id)` + `ORDER BY customer_id, date_of_lead`** keeps the first lead, the one that brought the customer in. Change to `DESC` to credit the latest lead.
- **`LEFT JOIN`** keeps depositors with no lead; they show `'Unknown'`.
- **`l.deleted_at`** carries its table alias. Unqualified, it fails with "column reference is ambiguous" once another joined table has the same column.
- **A top-level source** has no parent, so Level 1 shows `'Unknown'`. To count it as its own top level: `COALESCE(s2.name, s1.name, 'Unknown')`.

Checks

```
SELECT COUNT(*) AS rows, COUNT(DISTINCT customer_id) AS customers FROM (
<query> ) q;   -- must be equal
SELECT COUNT(*) FILTER (WHERE lead_id IS NULL) AS no_lead FROM ( <query> ) q;
```

Practice

- [ ] Count depositors and total deposits per `lead_source_1`, largest first.

3. pandas: who traded in October

Keep October BILL rows for the filtered customers, collect their IDs, and label each customer `Traded` or `Not Traded` with `isin` and `np.where`.

```
led_req = led[led['ClientID'].isin(cust_req)].copy()
led_req = led_req[led_req['LedgerType'] == 'BILL']
led_req = led_req[led_req['CreatedAt'].dt.to_period('M') == pd.Period('2026-
10', freq='M')]

customer_filtered = customer_filtered.copy()
traded_ids = set(led_req['ClientID'].astype(str).str.strip())
customer_filtered['Oct Trading'] = np.where(
    customer_filtered['Client ID'].astype(str).str.strip().isin(traded_ids),
'Traded', 'Not Traded')

customer_filtered['Oct Trading'].value_counts()
```

How it works

- **`.dt.to_period('M') == pd.Period('2026-10', freq='M')`** keeps one calendar month. The half-open range `(t >= '2026-10-01') & (t < '2026-11-01')` does the same.
- **`isin(traded_ids)`** gives True/False per customer; **`np.where(cond, a, b)`** turns that into labels.
- **`.astype(str).str.strip()` on both sides** stops spaces or number-versus-text IDs from hiding a match.
- **`.copy()`** after filtering avoids `SettingWithCopyWarning` when adding columns.
- **Case-insensitive status filter:** `~df['Status'].str.lower().isin(['demo', 'suspended'])` also catches `Demo`.
- **Moving a column:** `col = df.pop('Oct Trading')`, then `df.insert(position, 'Oct Trading', col)`.

Practice

- [ ] Add `Oct_Trading_Days` per customer (distinct dates of their October bills) next to the flag.

4. pandas: total and traded clients per Manager and agent

Count distinct Client IDs per group twice: once for everyone, once for an ID column that keeps only traders. `nunique` ignores blanks, so the second count is the traded clients.

```
cf = customer_filtered.copy()
cf['Assigned To'] = cf['Assigned To'].fillna('Unassigned')
cf['Traded_ID'] = cf['Client ID'].where(cf['Oct Trading'].eq('Traded'))   #
blank if not traded

pivot = (cf.groupby(['Manager', 'Assigned To'])
           .agg(Total_Clients=('Client ID', 'nunique'),
                Traded_in_Oct=('Traded_ID', 'nunique'))
           .reset_index())
pivot['Traded %'] = (pivot['Traded_in_Oct'] / pivot['Total_Clients'] *
100).round(1)
```

Subtotals and a grand total

```
mgr = (cf.groupby('Manager')
         .agg(Total_Clients=('Client ID', 'nunique'), Traded_in_Oct=
('Traded_ID', 'nunique'))
         .reset_index().assign(**{'Assigned To': 'Manager total'}))
grand = pd.DataFrame([{'Manager': 'All', 'Assigned To': 'Grand total',
                       'Total_Clients': cf['Client ID'].nunique(),
'Traded_in_Oct': cf['Traded_ID'].nunique()}])

report = pd.concat([pivot.drop(columns='Traded %').assign(_order=0),
mgr.assign(_order=1)])
report = report.sort_values(['Manager', '_order', 'Assigned To'],
kind='mergesort')
report = pd.concat([report, grand.assign(_order=2)],
ignore_index=True).drop(columns='_order')
report['Traded %'] = (report['Traded_in_Oct'] / report['Total_Clients'] *
100).round(1)
```

How it works

- **`.where(condition)`** keeps a value where the condition is True and blanks it elsewhere. Combined with `nunique`, it's a conditional distinct count with no `lambda`.
- **`nunique` instead of counting rows** means a client listed twice counts once.
- **`fillna('Unassigned')`** stops `groupby` silently dropping clients with no agent.
- **`_order`** is a helper column so each manager's agents come first and the `Manager total` row lands under them.

- **Percentages are worked out after adding the totals**, so the total rows get a correct percentage too, not an average of averages.

Practice

- [ ] Add `Not_Traded` and sort agents within each manager by `Traded %`, highest first.

5. SQL: calls after conversion, one row per client

Fix one conversion date per customer first, keep only calls after it, then aggregate per client with `FILTER` buckets and `STRING_AGG` lists. The first version joined `leads` directly, which counts every call once per lead.

```
WITH conv AS (                                    -- one conversion date per
customer
    SELECT customer_id, MIN(converted_at) AS converted_at
    FROM leads WHERE converted_at IS NOT NULL
    GROUP BY customer_id
),
calls_after AS (
    SELECT c.id AS customer_id, ctp.zynixx_client_id, cv.converted_at,
           cl.started_at, cl.answered_at,
           cl.started_at - cv.converted_at AS since_conversion
    FROM call_logs cl
    JOIN customers c                         ON c.id = cl.customer_id
    JOIN conv cv                             ON cv.customer_id = c.id
    LEFT JOIN customer_trading_profiles ctp  ON ctp.customer_id = c.id
    WHERE c.portal_id = {{snippet: portal_id}} AND c.project_id = {{snippet:
project_id}}
      AND cl.started_at > cv.converted_at
)
SELECT customer_id, zynixx_client_id,
       COUNT(*)                                                    AS
calls_after_conversion,
       COUNT(*) FILTER (WHERE since_conversion <= INTERVAL '1 day')   AS
within_1_day,
       COUNT(*) FILTER (WHERE since_conversion >  INTERVAL '30 days') AS
after_30_days,
       STRING_AGG(TO_CHAR(started_at AT TIME ZONE 'Asia/Kolkata', 'DD Mon YY
HH24:MI'), ', '
                  ORDER BY started_at)                             AS
call_times,
```

```
       COUNT(answered_at)                                                AS
answered_after_conversion
FROM calls_after
GROUP BY customer_id, zynixx_client_id;
```

How it works

- **`conv` with `MIN(converted_at)`** gives each customer exactly one conversion date, so `call_logs` rows can't multiply.
- **Compare raw, display converted.** `started_at > converted_at` uses stored times; `AT TIME ZONE` is only in the output.
- **`timestamp - timestamp` gives an interval**, compared with `INTERVAL '1 day'`, `'7 days'`, `'30 days'`.
- **`COUNT(*) FILTER (WHERE ...)`** counts only matching rows; several in one `SELECT` make the buckets in one pass.
- **`COUNT(*)` vs `COUNT(answered_at)`:** the first counts rows, the second counts non-blank values, so it's the answered calls.
- **`STRING_AGG(... ORDER BY started_at)`** lists values oldest first and skips NULLs, so an answered-times list holds only answered calls.
- **`customer_id` in `GROUP BY`** stops customers with no trading profile (blank `zynixx_client_id`) merging into one row.

Practice

- [ ] Add `days_to_first_answered`: days from conversion to the first answered call, using `MIN(answered_at) - converted_at`.

6. SQL: start from the ID list, not the calls

Turning the pasted IDs into a table with `UNNEST(ARRAY[...])` and starting the query from it gives every ID a row. With `WHERE id IN (...)`, an ID with no calls, or one that doesn't exist, simply vanishes.

```
WITH id_list AS (
    SELECT DISTINCT UNNEST(ARRAY[
        'S221954', 'P227319', 'A226436'          -- paste the full list
    ]) AS zynixx_client_id
),
```

```
ids AS (
    SELECT il.zynixx_client_id, ctp.customer_id,
           (c.portal_id = {{snippet: portal_id}} AND c.project_id =
{{snippet: project_id}}) AS in_scope
    FROM id_list il
    LEFT JOIN customer_trading_profiles ctp ON ctp.zynixx_client_id =
il.zynixx_client_id
    LEFT JOIN customers c                   ON c.id = ctp.customer_id
)
...
SELECT i.zynixx_client_id,
       CASE WHEN i.customer_id IS NULL    THEN 'ID not found'
            WHEN NOT i.in_scope           THEN 'Other portal/project'
            WHEN cv.converted_at IS NULL  THEN 'No conversion date'
            ELSE 'OK' END AS check_status,
       COALESCE(a.calls_after_conversion, 0) AS calls_after_conversion
FROM ids i
LEFT JOIN conv cv ON cv.customer_id = i.customer_id
LEFT JOIN agg a   ON a.zynixx_client_id = i.zynixx_client_id;
```

How it works

- **`UNNEST(ARRAY[...])`** turns a list into rows; the list keeps the same `'A', 'B'` format as an `IN (...)` list. `DISTINCT` drops IDs pasted twice.
- **`LEFT JOIN`s from the list** keep IDs that match nothing, so they can be labelled.
- **A boolean column, `in_scope`,** records whether the customer is in the portal and project, instead of filtering them out.
- **`check_status`** turns every zero into a reason: not found, wrong portal, no conversion date, or a real 0.
- **`COALESCE(..., 0)`** shows 0, not a blank, for clients with no calls.
- **Check:** the row count should equal the number of distinct IDs pasted.

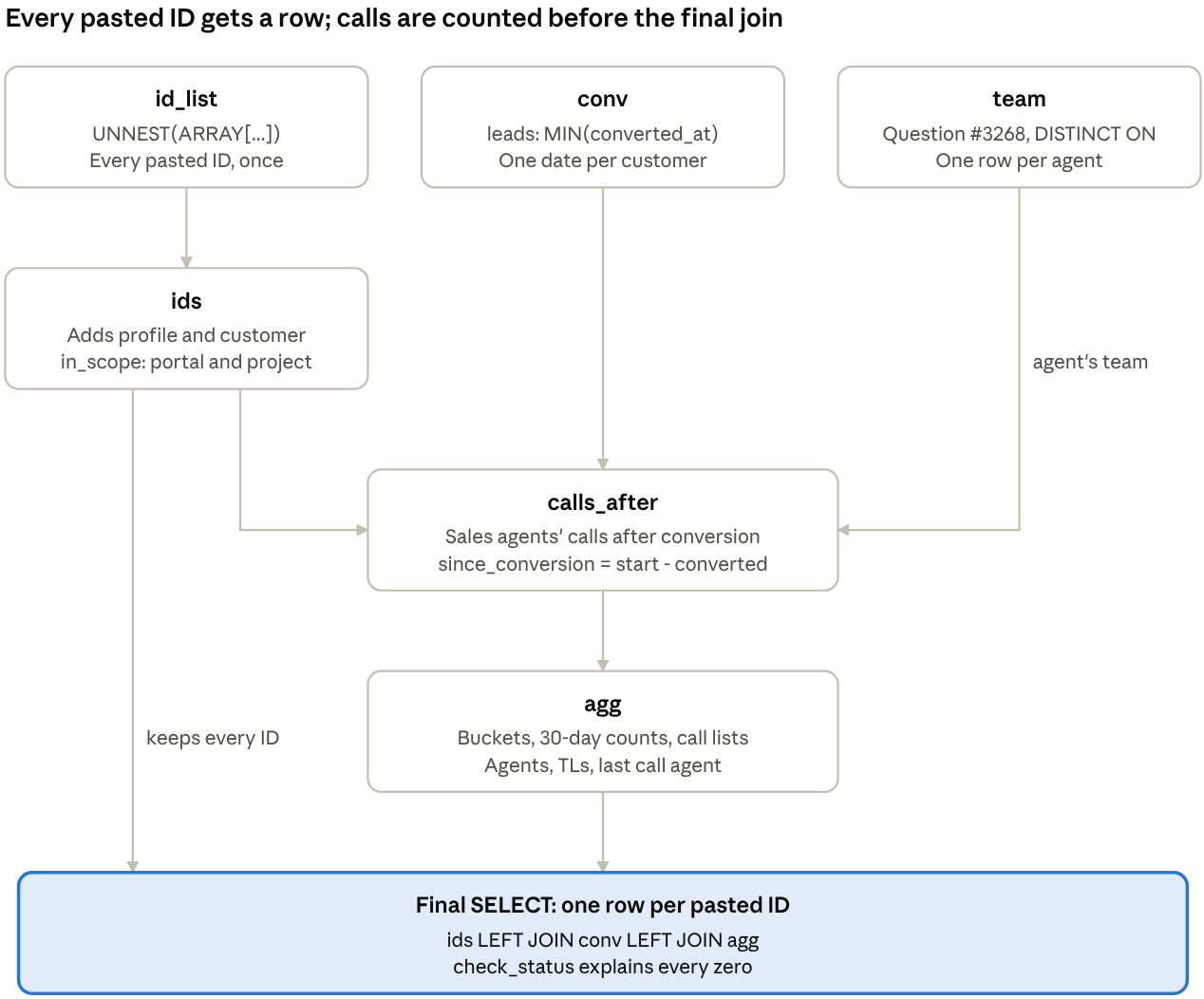
Pasting 1,000+ IDs by hand is fragile. If Metabase uploads are enabled, load the IDs as a CSV table and join to it instead.

Practice

- [ ] Count how many IDs land in each `check_status` value.

7. SQL: team list, Sales-only calls and 30-day counts

The final query adds the team list from Metabase question #3268, keeps only Sales agents' calls, and adds two 30-day columns beside the existing buckets.



calls after conversion · 6 CTEs into one final join

Lookups are cut to one row per customer or agent before they touch the calls, and the final join starts from the ID list, so no call repeats and no ID goes missing.

Joining a saved question safely

```
team AS (                                       -- one row per agent, so calls
can't repeat
    SELECT DISTINCT ON (user_id) user_id, agents, tl, manager, team
    FROM {{#3268-team-main}}
    ORDER BY user_id
),
```

Check the list first: `SELECT user_id, COUNT(*) FROM {{#3268-team-main}} GROUP BY user_id HAVING COUNT(*) > 1;` returns agents listed twice. `{{#3268-...}}` lets a native query use a saved question as if it were a table.

Keeping only Sales calls

```
SELECT team, COUNT(*) AS agents FROM {{#3268-team-main}} GROUP BY team ORDER
BY team;   -- exact values first

    JOIN team t ON t.user_id = cl.agent_user_id      -- JOIN, not LEFT JOIN
    WHERE ... AND t.team = 'Sales'
```

- **Filtering on a column of the joined table** removes calls by anyone else, including agents missing from the list.
- **`ILIKE 'sales%'`** catches `Sales Team` or `Sales - Dubai`, but check the values first so it doesn't catch `Sales Support`.

Lists and the latest agent

- **`STRING_AGG(DISTINCT agent, ', ' ORDER BY agent)`** lists each agent once. With `DISTINCT`, the `ORDER BY` must be the same expression.
- **`(ARRAY_AGG(agent ORDER BY started_at DESC))[1]`** is the agent on the most recent call; PostgreSQL arrays start at 1.
- **`TO_CHAR(...) || ' (' || agent || ')'`** attaches the agent's name to each call time in the list.

The 30-day columns

```
COUNT(*)           FILTER (WHERE since_conversion <= INTERVAL '30 days') AS
calls_within_30_days,
COUNT(answered_at) FILTER (WHERE since_conversion <= INTERVAL '30 days') AS
answered_within_30_days,
```

Check on every row: `calls_within_30_days = within_1_day + days_2_to_7 + days_8_to_30`.

Practice

- [ ] Add `answered_rate_30d` = answered within 30 days / calls within 30 days, guarding against division by zero with `NULLIF`.

8. Two misread requests: agree the columns first

The 30-day change took four rounds because the request was read two different wrong ways before the right one. Naming the exact columns to add, and what to keep, settles it in one round.

Round	Request	What was built	What was wanted
1	"only the calls within 30 days as days classification"	Day-by-day columns `day_0` to `day_30`	Two 30-day counts
2	"calls within 30 days, answered 30 days column only"	A stripped query with only those two counts	The two counts **added** to the full query
3	"also include them"	Lists and agents added back, 30 days only	The original buckets kept too
4	"consider the previous concept"	Two lines added to the full query	Correct

A request template that avoids this

```
Start from: <the query or file that's already right>
Add columns: <name> = <definition>, <name> = <definition>
Keep: everything else as it is
One row per: <client / agent / day>
```

For example: *Start from the full calls-after-conversion query. Add* `calls_within_30_days` *= Sales calls in the 30 days after conversion, and* `answered_within_30_days` *= answered ones in the same window. Keep everything else. One row per client.*

My side of it: when a request could mean "replace" or "add", I should ask which before rewriting a working query.

9. Metabase: Admin IDs that share an IP

The notebook editor can find IPs used by more than one Admin ID, but it can't list those IDs in one cell. For the list, convert the question to SQL and use `STRING_AGG`.

In the editor

1. Filter **IP → Not empty** (a blank IP row showed 10 admins; those are rows with no IP recorded).
2. Summarize **Number of distinct values of Admin ID** by **IP**.
3. Add a filter **after** Summarize: **Distinct values of Admin ID → greater than 1**.
4. To see the IDs for one IP, click its count, then **Break out by → Admin ID** or **View these Tdx Activity**.

Adding **Admin ID** as a second group gives one row per IP and Admin ID, but then each count is 1, so the "greater than 1" filter must go. Keeping only shared IPs that way needs a saved question joined back on IP.

In SQL (View the SQL → Convert this question to SQL)

```
SELECT ip,
       COUNT(DISTINCT admin_id)                                          AS
admin_count,
       STRING_AGG(DISTINCT admin_id::text, ', ' ORDER BY admin_id::text) AS
admin_ids
FROM tdx_activity
WHERE ip IS NOT NULL AND ip <> ''
GROUP BY ip
HAVING COUNT(DISTINCT admin_id) > 1
ORDER BY admin_count DESC;
```

- `HAVING` filters groups after counting; `WHERE` can't use `COUNT`.
- `::text` is needed because `STRING_AGG` joins text.
- **Converting to SQL** also shows the real table and column names behind Metabase's tidied ones.

Reading the result

The IPs start with `10.`, a private range: office network, VPN or a proxy. One IP with 19 admins is almost certainly a shared gateway, not one person. Matching on IP **and** `user_agent` together is a much stronger sign of the same device.

Practice

- [ ] Change the query to group by `ip` and `user_agent`, and compare how many groups still have more than one admin.

Problems and fixes

No error messages today; the issues were rows that would repeat, rows that would disappear, and requests read the wrong way.

Problem	Tool	Cause	Fix	Section
A depositor repeated per lead	SQL	Joining `leads` directly	`DISTINCT ON (customer_id)` in a CTE	2

Problem	Tool	Cause	Fix	Section
`deleted_at` could become ambiguous	SQL	No table alias	`l.deleted_at`	2
Calls counted once per lead	SQL	`leads` joined into the calls query	`conv` CTE with `MIN(converted_at)`	5
Clients with no profile merged into one row	SQL	Grouping only by `zynixx_client_id`	Add `customer_id` to `GROUP BY`	5
IDs with no calls missing	SQL	Starting from calls with `IN (...)`	Start from `UNNEST(ARRAY[...])` + `LEFT JOIN`s	6
Calls could repeat via the team list	SQL	An agent listed twice in #3268	`DISTINCT ON (user_id)`	7
Clients without an agent dropped from groups	pandas	`groupby` skips blank keys	`fillna('Unassigned')`	4
30-day request rebuilt three times	Process	Add vs replace not agreed	Name the columns and what to keep	8
No way to list Admin IDs per IP	Metabase	The editor has no list aggregation	Convert to SQL, `STRING_AGG`	9
A blank IP with 10 admins	Data	Activity rows with no IP	Filter IP not empty	9

Cheat sheet: new today

Only items not in the 29, 30 or 31 Oct cheat sheets.

SQL

Item	What it does	Example
`LEFT JOIN sources s2 ON s1.parent_id = s2.id`	The same table twice under two aliases	Source and parent source
`UNNEST(ARRAY['A', 'B'])`	A list into rows	Pasted client IDs
`{{#3268-team-main}}`	A Metabase saved question as a table	Team list
`{{snippet: portal_id}}`	A reusable Metabase SQL snippet	Portal and project filters
`(a = x AND b = y) AS in_scope`	A True/False column instead of a filter	Labelling rows
`timestamp - timestamp`	An interval	Time since conversion
`INTERVAL '30 days'`	A fixed length of time	Day buckets
`COUNT(column)`	Non-blank values only	Answered calls
`TO_CHAR(ts, 'DD Mon YY HH24:MI') || ' (' || agent || ')'`	Readable text from a time	Call lists
`HAVING COUNT(DISTINCT admin_id) > 1`	Groups with several distinct values	Shared IPs
`admin_id::text`	Cast a number to text	Before `STRING_AGG`

pandas

Item	What it does	Example
`pd.Period('2026-10', freq='M')`	A calendar month to compare with	October only
`s.where(cond)` + `nunique`	Distinct count of only some rows	Traded clients
`df.pop(col)` + `df.insert(i, col, values)`	Move a column	Flag next to the ID

Item	What it does	Example
`pd.concat([...]).sort_values([..., '_order'])`	Put subtotal rows under their group	Manager totals

Metabase

Item	What it does
Filter after Summarize	Filters the summarized result (like `HAVING`)
Click a value → Break out by	Splits one row by another column
View the SQL → Convert this question to SQL	Turns an editor question into editable SQL

Open items

The calls report needs two quick checks on the team list, and the workbook needs a first run.

- [ ] Run the team-list checks: agents listed twice, and the exact Sales value.
- [ ] Confirm on a few rows that `calls_within_30_days = within_1_day + days_2_to_7 + days_8_to_30`.
- [ ] Decide whether Lead Source 1 should fall back to the source itself when it has no parent.
- [ ] Load the 1,000+ client IDs as an uploaded table instead of pasting them, if Metabase uploads are enabled.
- [ ] Rerun the shared-IP check with IP + user agent.
- [ ] Mastery workbook exercises 1 to 15, then `progress()`.
- [ ] Still open from earlier: Uno API endpoint and a new token, the Uno connected rule, the first full October dialer run, and the settings-cell tidy-up.